Ejercicio 2 (1 punto). Sin definir otras funciones adicionales, implementar una función en lisp llamada **longitudes** que, dado un **número arbitrario de listas no anidadas**, devuelva sus longitudes en forma de lista.

(longitudes '(1 3 4 5)) → (4)

(longitudes '(1 3 4 5) '(13 4 3) '(2 3) NIL '(2 3 9 3)) → (4 3 2 0 4)

Ejercicio 3 (2 puntos). Sin definir otras funciones adicionales, implementar una función en lisp llamada **aplicarFunciones** que, dada **una lista sin anidamientos de valores** seguida de un **número arbitrario de funciones unarias**, devuelva una nueva lista del mismo tamaño que la lista de valores original, aplicando cada una de las funciones sobre los valores originales de forma sucesiva

(aplicarFunciones '(1 2 3 4) #'1+ (lambda(x) (* 2 x))) → (4 6 8 10)

(aplicarFunciones '(1 2 2 1) (lambda(x) (+ x 1)) (lambda(x) (- x 1))) → (1 2 2 1)

(aplicarFunciones '(1 2 2 1) (lambda(x) (+ x 1)) (lambda(x) (- x 1)) (lambda(x) (* x x))) → (1 4 4 1)

Ejercicio 4 (2 puntos). Sin definir otras funciones adicionales, implementar una función en lisp llamada **aplicarFuncion** que, dada **una función unaria y una lista** que pudiera ser o no anidada, aplique la función sobre cada elemento **respetando la estructura de la lista**.

(aplicarFuncion #'1+ '(1 (2 3 (3 4)) 4 (2 3))) → (2 (3 4 (4 5)) 5 (3 4)))

(aplicarFuncion (lambda(x) (* 2 x)) '(1 (2 2) 3)) → (2 (4 4) 6)

Aritméticas

(+ s_1 s_2 s_3 … s_n) (- s_1 s_2 s_3 … s_n)
(* s_1 s_2 s_3 … s_n) (/ s_1 s_2 s_3 … s_n)
(1+ s) (1- s)
(MAX s_1 s_2 s_3 … s_n) (MIN s_1 s_2 s_3 … s_n)
(FLOOR s_1) (CEIL s_1)
(MOD s_1 s_2)

Listas

(LIST e_1 e_2 … e_n) Devuelve una lista formada por los elementos especificados
(CAR lista) Retorna el primer elemento
(CDR lista) Retorna la cola de la lista
(CONS elemento lista) Devuelve nueva lista cuyo primer elemento es elemento y cuya cola es lista
(APPEND $lista_1$ $lista_2$ … $lista_n$) Concatenación de listas
(REVERSE lista) Invertir los elementos
(LENGTH lista) Devuelve el número de elementos
(REMOVE elemento lista) Borra el elemento dado de la lista

Comprobación de tipos

(ATOM arg) – ¿Es un átomo? T y NIL lo son
(LISTP arg) – ¿Es una lista?
(NUMBERP arg) – ¿Es un número?

Comprobación de valor

(ZEROP arg) – Devuelve T si arg es 0
(MINUSP arg) – Devuelve T si arg es negativo
(NULL arg) – Devuelve T si arg es NIL

Comparación de valores

(EQUAL arg_1 arg_2)
(MEMBER Elemento Lista)
(< arg_1 arg_2 arg_3 … arg_n)
(>arg_1 arg_2 arg_3 … arg_n)
(=arg_1 arg_2 arg_3 … arg_n)

Predicados lógicos

(NOT s)
(AND s_1 s_2 s_3 … s_n)
En el caso de evaluar todos a verdadero, devuelve el último valor obtenido (el de más a la derecha)
(OR s_1 s_2 s_3 … s_n)
En el caso de evaluar a verdadero, devuelve el primer valor distinto de NIL obtenido (el de más a la izquierda)

Estructuras condicionales

(IF Condicion ResultadoSiT ResultadoSiNoT)

(COND (Condicion1 Resultado1)
(Condicion2 Resultado2)
…
(Condicionn Resultadon))

Si ninguna condición evalúa a verdadero devuelve NIL
Para hacer condiciones exhaustivas, la $Condición_n$ puede ser T

Definición de funciones

(DEFUN NombreFuncion(s_1 s_2 s_3 … s_n) Descripción)

(lambda (x) (* 2 x))

Llamadas a funciones

(funcall *función* s_1 s_2 s_3 … s_n)
(apply *función* s_1 s_2 s_3 … s_n)
(*NombreFuncion* s_1 s_2 s_3 … s_n)

Obtención de función a partir del nombre:

#'nombre o FUNCTION nombre